Distributed Real Time Control Systems

MEEC

Report – Project 2nd Stage

Distributed Illuminance Control with Consensus ADMM

Authors:

Dinis Alves da Silva (106261) dinis.silva@tecnico.ulisboa.pt

Henrique Rodrigues (106362) henrique.d.sousa.rodrigues@tecnico.ulisboa.pt

2025/2026 – 2nd Semester, P3

1 System Architecture

1.1 Dual-Core Design and Inter-Core Communication

The RP2040 microcontroller features two ARM Cortex-M0+ cores that execute independently and concurrently. The firmware exploits this to impose a strict separation of concerns: **Core 0** is the real-time control core and **Core 1** is the CAN communication core. The two cores never share data structures directly; all information is exchanged through three hardware FIFOs implemented as lock-free queues provided by the Pico SDK (`pico/util/queue.h`).

Core 0 runs the 100 Hz control loop, executes the consensus ADMM optimisation, processes serial commands, and drives the `calibration_loop` state machine. Core 1 runs a tight polling loop that dequeues one outgoing frame from `can_tx_queue` per iteration, checks the MCP2515 interrupt line for incoming frames, reads up to two receive buffers (RXB0/RXB1) on each interrupt event, and pushes decoded frames into `can_rx_queue`. A third queue, `core1_ready_queue`, is used only once at boot: Core 0 posts a token to it after it has determined its own `LUMINAIRE` index, allowing Core 1 to complete its MCP2515 initialisation with the correct hardware-filter values. This handshake prevents Core 1 from configuring the CAN filters before the node identity is known.

The split architecture ensures that CAN interrupt latency is bounded and independent of the control-loop computation time. Because queue operations are non-blocking (`queue_try_add` / `queue_try_remove`), neither core can stall the other. A hardware interrupt handler on Core 1 sets a volatile flag `can_got_irq`; the Core 1 loop uses this flag together with a direct GPIO read to ensure no frame is missed even when the interrupt fires between iterations.

1.2 CAN Identifier Layout and Message Protocol

The MCP2515 controller implements the CAN 2.0A standard with 11-bit identifiers. The firmware packs all routing information into these 11 bits using the following field decomposition (Eq. 1):

$$\underbrace{\texttt{[10:8]}}_{\text{type (3 bits)}} \quad \underbrace{\texttt{[7:4]}}_{\text{dest (4 bits)}} \quad \underbrace{\texttt{[3:0]}}_{\text{src (4 bits)}} \tag{1}$$

The *type* field (3 bits, values 0–7) carries the message class: `MSG_WAKEUP` (0x0), `MSG_CAL` (0x1), `MSG_SET` (0x2), `MSG_GET` (0x3), `MSG_REPLY` (0x4), `MSG_ADMM` (0x6), and `MSG_CTRL` (0x7). The *dest* field encodes either a unicast luminaire index (1–3) or the broadcast address (0x0). The *src* field carries the sender's luminaire index. All three helpers (`MAKE_CAN_ID`, `CAN_ID_TYPE`, `CAN_ID_DEST`, `CAN_ID_SRC`) are implemented as preprocessor macros to avoid any runtime overhead.

The MCP2515 acceptance filters are programmed at startup to reject all frames whose *dest* field does not match either the node's own index or the broadcast address 0x0. The mask `0x0F0` isolates bits [7:4], and two filter pairs cover the unicast and broadcast cases respectively. This offloads frame-level routing entirely to hardware, keeping the Core 1 receive loop free of software filtering overhead.

Frame payload conventions are uniform: `data[0]` always carries a *sub-type* byte that further qualifies the message within its class, and floating-point values are transmitted as raw IEEE-754 little-endian bytes in `data[1..4]`. Three inline helper functions

(`can_send_float`, `can_send_byte`, `can_send_sub`) construct frames and enqueue them in a single call, ensuring consistent encoding throughout the firmware.

1.3 Boot Sequence, Node Discovery and Distributed Calibration

Each node determines its own `LUMINAIRE` index by reading the unique 64-bit flash identifier burnt into the RP2040 at manufacture; the least significant byte is looked up in a compile-time table (`UID_TABLE`). This makes node assignment permanent and requires no manual DIP switches or jumpers.

After identity resolution, nodes enter a *wakeup* phase in which each node periodically broadcasts a `SYN` wakeup message and replies with `SYN_ACK` to any received `SYN`. A node exits the wakeup phase only when it has exchanged at least one wakeup message with each of the remaining $N-1$ peers, guaranteeing that all nodes are reachable before the calibration barrier is armed.

Distributed calibration proceeds as a *round-robin* over all nodes. A synchronisation barrier prevents any node from starting until all peers have declared themselves ready (`SUB_CAL_READY`). In each round, one node (the *active* node) turns on its LED at full duty cycle, waits for the illuminance to settle (3 s), and measures its own sensor response to obtain the self-gain k_{ii}. All other (*passive*) nodes simultaneously measure the cross-coupling k_{ji} and report their readings back to the active node via a unicast `SUB_CAL_LUX` message. The active node then broadcasts `SUB_CAL_DONE` and the round advances. After all N rounds complete, each node possesses the full $N \times N$ coupling-gain matrix $\mathbf{K}$ locally, which is the only input required by the ADMM optimiser.

2 Distributed Optimisation

2.1 Control Objective and Problem Formulation

Part 1 treated r_i as an exact target; the PI controller's sole goal was to drive the tracking error to zero as fast as possible. Part 2 changes the objective fundamentally: r_i is replaced by an *illuminance lower bound* L_i (determined by the occupancy state of desk i), and a linear energy cost is introduced. The network-wide optimisation problem is

$$\min_{\mathbf{u}} \sum_{i=1}^{N} c_i u_i \quad \text{s.t.} \quad \mathbf{k}_i^T \mathbf{u} + d_i \geq L_i,\ 0 \leq u_i \leq 1, \quad \forall\, i, \tag{2}$$

where $u_i \in [0,1]$ is the duty cycle of luminaire i, $c_i \geq 0$ is its energy cost, $\mathbf{k}_i \in \mathbb{R}^N$ is the row of the coupling-gain matrix corresponding to desk i (with k_{ij} denoting the illuminance contributed at desk i by LED j), and d_i is the background illuminance at desk i. The coupling between desks means that a node with a low cost can intentionally over-illuminate a neighbour, thereby allowing an expensive neighbour to reduce its own duty cycle. This trade-off is what centralised and greedy controllers cannot exploit, and is the primary motivation for a distributed optimisation layer.

2.2 Why Consensus ADMM?

Three distributed approaches of increasing capability are presented in the course modules, and each one fixes the limitation of the previous:

Non-coordinated control (Module 15) has no inter-node communication. Each node computes $u_i = (r_i - d_i)/k_{ii}$ using only its own gain and reference. Cross-coupling from neighbours is treated as an unknown disturbance absorbed by the PI controller. This is correct only when coupling gains are small ($k_{12}k_{21} < k_{11}k_{22}$); with strong coupling the nodes fight each other, causing oscillations and energy waste.

Coordinated feedforward (Module 16) fixes coupling by broadcasting duty values over CAN so each node can subtract the expected cross-contribution before computing its own duty. Stability improves, but the method is inherently *selfish*: every node minimises its own tracking error without regard for the network-wide cost. There is no energy-minimisation objective.

Dual decomposition (Module 18) introduces the cost function $\sum c_i u_i$ and decomposes the global problem into per-node sub-problems via Lagrangian relaxation, exchanging dual variables λ_i over CAN. However, the primal sub-problem gradient is c_i, a constant independent of u_i. Setting it to zero yields no solution; the primal update is ill-posed for linear costs, causing the algorithm to diverge.

Consensus ADMM (Module 19) resolves this by augmenting the Lagrangian with a quadratic penalty term $(\rho/2)\|\mathbf{u} - \bar{\mathbf{u}}\|^2$, creating a strictly convex bowl around the consensus point $\bar{\mathbf{u}}$. This makes the primal update uniquely solvable in closed form even with a linear cost, while preserving convergence to the global optimum of (2).

2.3 Consensus ADMM Algorithm

Each node i maintains a local primal vector $\mathbf{x}_i \in \mathbb{R}^N$ (its proposal for the duty cycle of *all* nodes), a consensus average $\bar{\mathbf{u}} \in \mathbb{R}^N$, and a dual variable vector $\boldsymbol{\lambda}_i \in \mathbb{R}^N$. One iteration consists of three coupled updates.

Primal (x) update. Node i solves the local constrained QP

$$\mathbf{x}_i^{(k+1)} = \arg\min_{\mathbf{u}\in\mathcal{C}_i} \left\{ \mathbf{c}_i^T \mathbf{u} + \boldsymbol{\lambda}_i^{(k)\,T} \mathbf{u} + \tfrac{\rho}{2} \left\| \mathbf{u} - \bar{\mathbf{u}}^{(k)} \right\|^2 \right\}, \tag{3}$$

where the local feasible set is $\mathcal{C}_i = \{\mathbf{u} : \mathbf{k}_i^T \mathbf{u} \geq L_i - d_i,\ 0 \leq u_i \leq 1\}$, and $\mathbf{c}_i$ is the sparse cost vector (nonzero only at component i). Completing the square in (3) yields an equivalent unconstrained minimiser at $\mathbf{u}^* = \rho^{-1}\mathbf{z}_i$ with $\mathbf{z}_i = \rho\bar{\mathbf{u}}^{(k)} - \mathbf{c}_i - \boldsymbol{\lambda}_i^{(k)}$. If this unconstrained point is feasible it is the global solution. Otherwise the constrained solution lies on a boundary of $\mathcal{C}_i$, and there are exactly six candidates (one for each face and edge of the three active constraints); they are evaluated analytically and the cheapest feasible one is selected.

Consensus ($\bar{\mathbf{u}}$) update. A key result of consensus ADMM is that the sum $\sum_i \boldsymbol{\lambda}_i^{(k)} = \mathbf{0}$ is invariant under the dual update. This reduces the z-update to a plain average:

$$\bar{u}_j^{(k+1)} = \frac{1}{N} \sum_{i=1}^{N} x_{ij}^{(k+1)}. \tag{4}$$

Each node only needs to broadcast its own component x_{ii} (the duty it proposes for itself, the only one it actually controls) via a `MSG_ADMM/SUB_ADMM_U_OPT` message. No dual-variable broadcast is needed; the consensus average reduces to setting $\bar{u}_j^{(k+1)} = x_{jj}^{(k+1)}$ (the value node j just broadcast).

Dual ($\boldsymbol{\lambda}$) update.

$$\boldsymbol{\lambda}_i^{(k+1)} = \boldsymbol{\lambda}_i^{(k)} + \rho \left(\mathbf{x}_i^{(k+1)} - \bar{\mathbf{u}}^{(k+1)} \right). \tag{5}$$

The residual $\mathbf{x}_i - \bar{\mathbf{u}}$ measures the disagreement between node i's proposal and the network consensus; $\boldsymbol{\lambda}_i$ accumulates this disagreement and penalises future primal updates that deviate from consensus. Convergence is declared when $|x_{ii}^{(k+1)} - \bar{u}_i^{(k+1)}| < \varepsilon = 10^{-4}$, with a hard cap of 20 iterations and a per-iteration CAN timeout of 50 ms.

2.4 Integration with the PI Controller

ADMM and the PI controller operate on completely different timescales and serve complementary roles. ADMM runs *once per event* (occupancy change or cost update), computes the globally optimal duty cycle u_i^* for this node, and converts it to a lux reference via $r_i = d_i + k_{ii} u_i^*$. The PI controller then runs every 10 ms continuously, tracking this reference against real LDR measurements and correcting for disturbances (sunlight, shadows, thermal drift) that ADMM cannot anticipate. The ADMM result is warm-started from the feedforward estimate $u_{\text{ff}} = (L_i - d_i)/G_i$ to reduce the number of iterations needed when the system is near its previous operating point.

3 Results

Tables and figures showing measured steady-state lux values, duty cycles, and performance metrics (energy E, cumulative visibility error V, flicker F) for the three-node network under different occupancy configurations and cost vectors will be presented in the final report. Preliminary experiments confirm that the ADMM algorithm converges in 3–6 iterations for the tested configurations and that the PI controller tracks the ADMM-derived reference with the same settling time and zero steady-state error characterised in Part 1.

4 Attachments

4.1 Key Equations

CAN identifier packing. The 11-bit CAN identifier is divided into three fields: message type (bits 10–8), destination address (bits 7–4), and source address (bits 3–0).

$$\texttt{ID} = (\text{type} \,\&\, 0x7) \ll 8 \mid (\text{dest} \,\&\, \texttt{0xF}) \ll 4 \mid (\text{src} \,\&\, \texttt{0xF}) \tag{6}$$

ADMM $\mathbf{z}_i$ vector. The intermediate vector used to form the QP in closed form:

$$z_{ij}^{(k)} = \rho\, \bar{u}_j^{(k)} - c_{ij} - \lambda_{ij}^{(k)}, \qquad j = 1, \ldots, N. \tag{7}$$

Unconstrained minimiser. The unique minimiser of the augmented Lagrangian ignoring constraints:

$$u_{ij}^{*} = \frac{z_{ij}^{(k)}}{\rho} = \bar{u}_j^{(k)} - \frac{\lambda_{ij}^{(k)}}{\rho} - \frac{c_{ij}}{\rho}. \tag{8}$$

Boundary candidate — illuminance constraint active (C1). If the illuminance constraint $\mathbf{k}_i^T \mathbf{u} = L_i - d_i$ is tight:

$$u_{ij}^{\mathrm{C1}} = \frac{z_{ij}^{(k)}}{\rho} - k_{ij} \cdot \frac{d_i - L_i + \rho^{-1} \mathbf{k}_i^T \mathbf{z}_i^{(k)}}{\|\mathbf{k}_i\|^2}. \tag{9}$$

Boundary candidates — LED bounds active (C2, C3). Clamping the own duty at its lower or upper bound:

$$u_{ij}^{\mathrm{C2}} = \frac{z_{ij}^{(k)}}{\rho}, \quad u_{ii}^{\mathrm{C2}} = 0; \qquad u_{ij}^{\mathrm{C3}} = \frac{z_{ij}^{(k)}}{\rho}, \quad u_{ii}^{\mathrm{C3}} = 1. \tag{10}$$

Corner candidates — two constraints simultaneously active (C1∩C2 and C1∩C3). Let $m^2 = \|\mathbf{k}_i\|^2 - k_{ii}^2$. Then for $j \neq i$:

$$u_{ij}^{\mathrm{C12}} = \frac{z_{ij}^{(k)}}{\rho} - k_{ij} \cdot \frac{d_i - L_i + \rho^{-1}(\mathbf{k}_i^T \mathbf{z}_i^{(k)} - k_{ii}\, z_{ii}^{(k)})}{m^2}, \quad u_{ii}^{\mathrm{C12}} = 0, \tag{11}$$

$$u_{ij}^{\mathrm{C13}} = \frac{z_{ij}^{(k)}}{\rho} - k_{ij} \cdot \frac{d_i - L_i + k_{ii} + \rho^{-1}(\mathbf{k}_i^T \mathbf{z}_i^{(k)} - k_{ii}\, z_{ii}^{(k)})}{m^2}, \quad u_{ii}^{\mathrm{C13}} = 1. \tag{12}$$

Reference conversion from ADMM result. The optimal duty u_i^* is translated to a PI lux reference as:

$$r_i = d_i + k_{ii}\, u_i^*. \tag{13}$$

Performance metrics. Over T samples at period h:

$$E = \sum_k u_k \cdot P_{\max} \cdot h, \tag{14}$$

$$V = \frac{1}{T} \sum_k \max(0,\, L - \hat{x}_k), \tag{15}$$

$$F = \frac{1}{T \cdot h} \sum_{k:\Delta u_k \cdot \Delta u_{k-1} < 0} \Big(|\Delta u_k| + |\Delta u_{k-1}| \Big), \tag{16}$$

where $\hat{x}_k = d + G u_k$ is the duty-cycle-based illuminance estimate used for visibility error (as specified in the project guide), $\Delta u_k = u_k - u_{k-1}$, and the flicker sum is conditioned on sign reversal of consecutive duty-cycle increments.

4.2 Figures

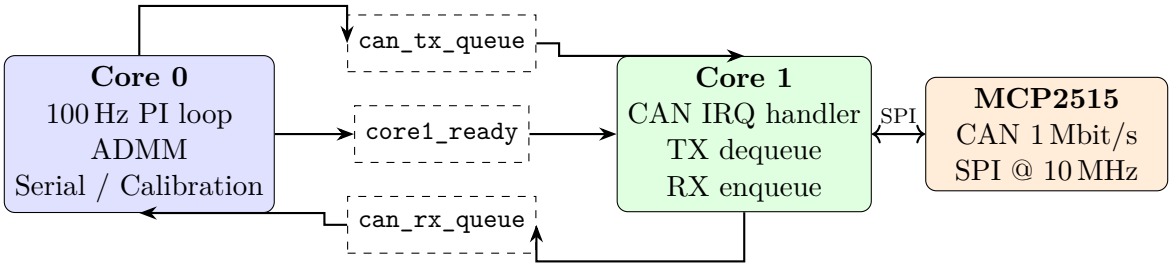

Figure 1: Dual-core firmware architecture. Core 0 enqueues outgoing frames into `can_tx_queue` and dequeues received frames from `can_rx_queue`. Core 1 services the MCP2515 interrupt and moves frames between the queues and the hardware. A one-shot `core1_ready` queue synchronises boot order.

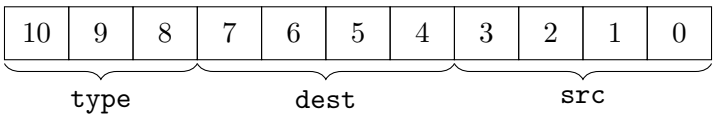

Figure 2: 11-bit CAN identifier field decomposition. Bits [10:8] carry the message-type class (8 possible classes), bits [7:4] carry the destination luminaire index (0 = broadcast, 1–3 = unicast), and bits [3:0] carry the source luminaire index.

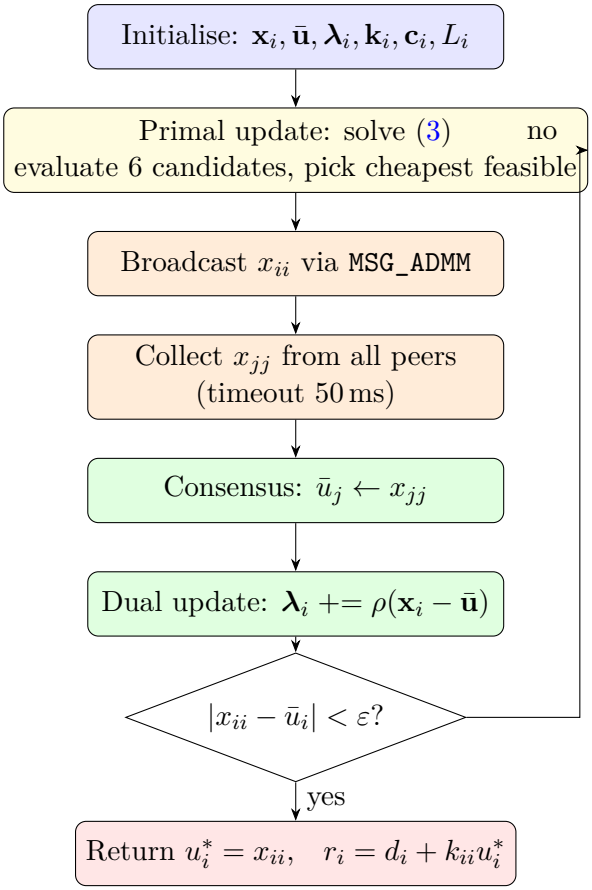

Figure 3: One ADMM optimisation run. The loop iterates until the primal residual on the local component falls below $\varepsilon = 10^{-4}$ or 20 iterations elapse. Only x_{ii} is exchanged over CAN per iteration; no dual-variable broadcast is necessary because $\sum_i \boldsymbol{\lambda}_i = \mathbf{0}$.

4.3 Tables

Table 1: ADMM algorithm parameters.

Parameter	Value	Description
ρ	0.07	ADMM penalty parameter; controls convergence speed vs. accuracy trade-off
ε	10^{-4}	Primal residual convergence threshold on own component $\lvert x_{ii} - \bar{u}_i \rvert$
$N_{\max}$	20	Maximum number of ADMM iterations per optimisation run
T_{CAN}	50 ms	Per-iteration timeout waiting for peer broadcasts
N	3	Number of luminaire nodes in the network

Table 2: CAN message types and sub-types implemented in the firmware.

Type	**Sub-type**	**Payload and purpose**
`MSG_WAKEUP` (0x0)	`SUB_SYN` / `SUB_SYN_ACK`	Broadcast during boot; used to discover all peers before calibration

`MSG_CAL` (0x1)	`SUB_CAL_READY`
	`SUB_CAL_ON`
	`SUB_CAL_LUX`
	
	`SUB_CAL_DONE`
		Node declares itself ready to calibrate
		Active node signals LED fully on
		Passive node reports measured lux (float, 4 bytes)
		Active node signals end of its round

`MSG_SET` (0x2)	`'r','o','u','f',...`	Set reference, occupancy, duty, feedback mode, costs
`MSG_GET` (0x3)	`'y','u','r','E','V','F',...`	Request a variable; reply arrives in `MSG_REPLY`
`MSG_REPLY` (0x4)	same as GET sub-type	Float response to a GET query
`MSG_ADMM` (0x6)	`SUB_ADMM_U_OPT`	Per-iteration broadcast of own optimal duty x_{ii} (float, 4 bytes)
`MSG_CTRL` (0x7)	`SUB_ACK` / `SUB_REBOOT`	Acknowledgement and network-wide reboot